Software Project Concept Notes

BİL481 Software Engineering Project — Academic Year 2026

1. Project Name

- **Project Working Title:** Green Software Metrics Framework (GSMF)
- **Group Name:** Intellectual Property
- **Team Members:**
    - Reşat Efe Koca (Student ID: 231301010)
    - Bülent Çakır (Student ID: 231301064)
    - Murat Yılmaz (Student ID: 241101063)
    - Furkan Yaman (Student ID: 221301015)

2. Short Description

The Green Software Metrics Framework (GSMF) is an empirical evaluation platform and benchmarking harness engineered to quantify, compare, and report the runtime energy consumption, execution efficiency, and operational carbon footprint of applications deployed on No-Code / Low-Code (NC/LC) platforms versus custom hand-coded equivalents.

While low-code platforms drastically reduce development cycles, the runtime overhead of their visual engine abstractions and background microservice dependencies remains unmeasured. Because hardware energy counters cannot profile remote vendor-hosted cloud servers, GSMF evaluates self-hosted, containerized low-code stacks—specifically Budibase and Appsmith deployed via Docker—and benchmarks them against a hand-coded Python FastAPI baseline. All three targets implement an identical standardized reference application featuring a customer management workload: record creation, paginated querying, and batch updating.

GSMF operationalizes the Green Software Foundation (GSF) Software Carbon Intensity (SCI) specification for operational emissions, energy per functional unit, and the academic Energy-Delay Product (EDP). By executing with strict temporal isolation (one platform stack per run on a controlled Linux host), offloading load generation (k6) to a secondary machine, applying DVFS/thermal controls from literature (Pereira et al., 2017), and sampling direct kernel RAPL sysfs counters, GSMF provides statistically sound, reproducible green software metrics.

3. Requirements (Functional & Non-Functional)

1. **FR-1 (Standardized Reference Workload Execution):** The system shall implement an identical reference CRUD application schema (item creation, paginated listing, and batch updates) across self-hosted Docker targets (Budibase, Appsmith) and a hand-coded FastAPI baseline.
2. **FR-2 (Two-Host Distributed Orchestration):** The test harness must orchestrate benchmarks over a local network where the load generator (k6) runs on a secondary machine to prevent load client CPU cycles from corrupting target energy counters.
3. **FR-3 (Strict Temporal Isolation & Lifecycle Automation):** The runner shall execute exactly one container stack at a time (docker compose up/down), ensuring zero

inter-platform interference and clean system state.

4. **FR-4 (Idle Baseline Calibration & Net Energy Derivation):** The framework shall sample 60 seconds of system idle power prior to each benchmark iteration and derive net dynamic energy by subtracting idle baseline draw (P_idle * delta_t_run) from gross hardware RAPL energy.
5. **FR-5 (Functional Unit Normalization & Operational SCI):** The system shall calculate energy per functional unit R (defined as one complete user task: 1 create + 1 paginated list + 1 batch update) and compute the operational SCI score in gCO2eq per 1,000 completed user tasks: SCI_op = (E * I * 1000) / R, using a cited regional grid carbon intensity for Türkiye (embodied carbon M documented as future work).
6. **FR-6 (Multi-Trial Statistical Aggregation):** The engine shall execute a minimum of 10 identical iterations per target and compute the mean, median, standard deviation, and 95% confidence intervals for Total Energy (Joules), Execution Time (seconds), and Energy-Delay Product (EDP = E * T).
7. **FR-7 (Secondary Diagnostic Profiling & Optional GSF Export):** The system shall optionally collect per-container CPU/memory resource shares using Scaphandre as a secondary diagnostic breakdown and support exporting run records to GSF Impact Framework-compliant YAML manifests.
8. **NFR-1 (Experimental Rigor & Thermal Control):** In accordance with Pereira et al. (2017), the testbed host shall enforce disabled Intel Turbo Boost, pinned CPU governor (performance), and a mandatory 30–60 second cooling rest period between iterations to eliminate DVFS/thermal throttling variations.
9. **NFR-2 (Direct Hardware Access without Metering Devices):** Energy measurements shall be captured directly from Linux sysfs kernel powercap interfaces (/sys/class/powercap/intel-rapl) on standard x86 hardware, maintaining zero hardware acquisition costs.
10. **NFR-3 (CLI-First Architecture & Lightweight Footprint):** The primary evaluation harness shall operate entirely as a self-contained Python CLI pipeline; telemetry collection routines must maintain under 2% CPU overhead on the test host.

4. Simple Pseudocode for Implementation Parts

Note: All values shown below are illustrative example values, not measured results.

5. Sample Views

```
None
FUNCTION executePlatformBenchmark(platformTarget, workloadPlan):
    // Inputs: target config {name, composePath, url}, workload {tasks, concurrency, reps=10, cooldownSec=45}
    setCPUGovernor("performance"); disableTurboBoost()
    iterationResults = []
    FOR run = 1 TO workloadPlan.reps DO:
```

```
        dockerComposeUp(platformTarget.composePath);
waitForHealthCheck(platformTarget.url)
        idleWatts = (readRAPLEnergy() after 60s - readRAPLEnergy()) / 60.0
        triggerRemoteWarmup(platformTarget.url, tasks=50)
        startE = readRAPLEnergy(); startT = now()
        k6Status = triggerRemoteLoad(runnerHost="192.168.1.100", target=platformTarget.url,
tasks=workloadPlan.tasks)
        duration = now() - startT; grossE = readRAPLEnergy() - startE
        netE = MAX(0.0, grossE - (idleWatts * duration))
        iterationResults.append({netE: netE, duration: duration, edp: netE * duration})
        dockerComposeDown(platformTarget.composePath);
SLEEP(workloadPlan.cooldownSec)
    END FOR
    avgE = mean(iterationResults.netE); ci95E = confidenceInterval95(iterationResults.netE)
    avgT = mean(iterationResults.duration); avgEDP = mean(iterationResults.edp)
    sciOperationalPerKTasks = ( (avgE / 3.6e6) * 440.0 * 1000.0 ) / workloadPlan.tasks
    // Output: aggregated metrics (mean, 95% CI, duration, EDP, SCI score)
    return {avgE, ci95E, avgT, avgEDP, sciOperationalPerKTasks}
END FUNCTION
```

View 1: CLI Execution & Summary Terminal

View 2: Lightweight Comparative Web Dashboard

```
None
================================================================================
================================
                GSMF EXPERIMENT SUMMARY RESULTS (N=10)
                [Illustrative example values, not measured results]
================================================================================
================================
Target       | Net Energy (J)  | 95% CI (J)    | Time (s) | EDP (J*s)   | SCI (gCO2eq/kTask)
-------------+-----------------+---------------+----------+-------------+--------------------------
FastAPI      |   418.52 J      | ± 8.14 J      |  18.10 s |   7,575.2   | 0.0511 gCO2eq/kTask
Appsmith     | 1,280.40 J      | ± 24.60 J     |  33.40 s |  42,765.3   | 0.1564 gCO2eq/kTask
```

```
Budibase     | 1,862.10 J      | ± 38.25 J     |  43.20 s |  80,442.7   | 0.2275 gCO2eq/kTask
=================================================================================================
```

6. Sample Design Information (Architecture, Tech Stack, & Data Models)

```
None
+-----------------------------------------------------------------------------------------------+
|  GSMF  ::  Low-Code vs Hand-Coded Sustainability Dashboard                    [Export GSF YAML] |
+-----------------------------------------------------------------------------------------------+
|  Workload: 1,000 Standard CRUD User Tasks (N=10 Trials, 95% Confidence Intervals)             |
|                                                                                               |
|  [Illustrative example values, not measured results]                                          |
+-----------------------------------------------------------------------------------------------+
|  Platform     | Net Energy (J) | Execution Time | EDP (J*s)   | Operational SCI (gCO2eq/kTask)    |
|  -------------+----------------+----------------+-------------+-----------------------------------|
|  FastAPI      | 418.5 J ± 8.1  | 18.1 s ± 0.4   | 7,575 J*s   | 0.051 gCO2eq/kTask                |
|  Appsmith     | 1280.4 J ± 24.6| 33.4 s ± 0.9   | 42,765 J*s  | 0.156 gCO2eq/kTask                |
|  Budibase     | 1862.1 J ± 38.2| 43.2 s ± 1.2   | 80,442 J*s  | 0.227 gCO2eq/kTask                |
+-----------------------------------------------------------------------------------------------+
|  [Bar Chart: Energy per User Task]                  [Bar Chart: Energy-Delay Product (EDP)]   |
|                                                                                               |
|  FastAPI  [###] 0.42 J/task                         FastAPI  [##] 7.5k                        |
|  Appsmith [##########] 1.28 J/task                  Appsmith [###########] 42.7k              |
|  Budibase [###############] 1.86 J/task             Budibase [###################] 80.4k     |
|                                                                                               |
+-----------------------------------------------------------------------------------------------+
```

6.1 Architecture Overview

- **Target Testbed Host:** Headless Ubuntu Desktop / Mini PC running isolated target

container stacks, enforcing CPU governor/thermal policies, sampling Linux RAPL sysfs counters, and hosting the GSMF CLI daemon.
- **Load Generator Host:** Secondary laptop/client running k6 over a local network, firing standardized CRUD batches.
- **Analysis & Reporting Service:** Python aggregation service computing statistical metrics (mean, 95% CI, EDP, operational SCI per 1,000 tasks) and generating CSV/GSF YAML reports.

6.2 Tech Stack Selection

6.3 Data Models

- **TargetPlatform:** id (UUID), name (String), composeFilePath (String), endpointBaseUrl (String)

Layer	Selected Technology	Rationale & Decision Alignment
Evaluation Targets	Budibase (Docker), Appsmith (Docker), FastAPI + SQLite Baseline	Self-hosted open-source stacks enabling direct Linux RAPL access; identical CRUD schema.
Telemetry Core	Python 3.11 CLI + Linux RAPL sysfs (/sys/class/powercap/intel-rapl)	Direct MSR energy access, zero hardware cost, <1.5% CPU overhead.
Diagnostic Profiler	Scaphandre	Secondary diagnostic only for container cgroup CPU-share breakdowns.
Load Generator	k6 (on Secondary Host)	High-precision virtual user execution over network without corrupting testbed RAPL counters.
Reporting & UI	FastAPI + Jinja2 / Chart.js (CLI-first)	Lightweight statistical tables and charts without heavy frontend scope creep.

-
- **ExperimentSuiteConfig:** id (UUID), title (String), totalUserTasks (Integer), concurrency (Integer), repetitions (Integer), cooldownSeconds (Integer), gridCarbonIntensity (Float)
- **IterationMeasurement:** id (Integer), suiteId (FK), targetId (FK), iterationIndex (Integer), idlePowerWatts (Float), grossEnergyJoules (Float), netEnergyJoules (Float), durationSeconds (Float), edpJouleSeconds (Float), p95LatencyMs (Float)
- **BenchmarkSummaryResult:** id (UUID), suiteId (FK), targetId (FK),

meanNetEnergyJoules (Float), ci95EnergyJoules (Float), meanDurationSeconds (Float), meanEDP (Float), operationalSCIScorePerKTasks (Float)

7. Scenarios (User Workflows, Preconditions, & Postconditions)

Scenario 1: Automated Comparative Benchmarking Pipeline Execution

- **Preconditions:** Headless Linux testbed with RAPL access; Docker compose stacks ready; secondary k6 machine connected on local network.
- **User Workflow:**
  1. Researcher runs gsmf-benchmark run --config experiments/crud_suite.yaml on testbed.
  2. GSMF automatically sets CPU frequency governors to performance and disables Turbo Boost.
  3. GSMF starts FastAPI stack, measures 60s idle baseline, runs 10 k6 iterations with 45s cooldowns, and records RAPL deltas.
  4. GSMF repeats the identical protocol for Appsmith, then Budibase.
  5. GSMF computes statistical aggregations (mean, 95% CI, EDP, operational SCI per 1,000 tasks).
- **Postconditions:** Results saved to SQLite; CSV and terminal summary tables output.

Scenario 2: Sustainability Evaluation & GSF Impact Export

- **Preconditions:** Completed benchmark runs exist in database.
- **User Workflow:**
  1. User runs gsmf-report --format gsf-yaml --suite-id <ID>.
  2. GSMF calculates operational SCI per 1,000 tasks using regional grid carbon intensity (440 gCO2eq/kWh).
  3. System generates GSF Impact Framework-compliant YAML manifest.
- **Postconditions:** Validated GSF YAML file saved for verification and carbon reporting.

8. References

- Pereira et al. (2017): Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. "Energy efficiency across programming languages: how do energy, time, and memory relate?" Proc. 10th ACM SIGPLAN SLE 2017, pp. 256–267. https://dl.acm.org/doi/10.1145/3136014.3136031
- Green Software Foundation (GSF) SCI Specification: Software Carbon Intensity Specification v1.0. https://grnsft.org/sci
- Green Software Foundation (GSF) Impact Framework: https://github.com/Green-Software-Foundation/if
- Scaphandre: Energy consumption metrology agent. https://github.com/hubblo-org/scaphandre
- Cloud Carbon Footprint (CCF): Methodology & Architecture. https://www.cloudcarbonfootprint.org/docs/methodology/
- CodeCarbon: Tracking operational carbon emissions. https://github.com/mlco2/codecarbon

- Türkiye Grid Carbon Intensity: World Bank Group & Our World in Data (Electricity grid carbon intensity benchmark for Türkiye: ~440–500 gCO2eq/kWh average grid emission factor). https://ourworldindata.org/profile/co2/turkey